



SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Code of Conduct:

I P.Ranjithkumar (192411119)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Ranjithkumar

1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Testing should be treated as a systematic engineering activity, not random execution of test cases. The team should analyze requirements, identify risks, design test cases using techniques such as equivalence partitioning and boundary value analysis, execute them, and document defects. Structured test design provides better test coverage, repeatability, and defect detection. For example, instead of randomly testing a login page, the tester tests valid, invalid, empty, boundary-length, and special-character inputs.

2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

A well-defined testing process includes requirement analysis, test planning, test case design, execution, defect reporting, retesting, and regression testing. In a banking system, transactions such as deposit, withdrawal, fund transfer, and balance checking must be systematically tested. For example, a withdrawal test should verify sufficient balance, insufficient balance, invalid amount, and transaction limits.

This process improves reliability, security, accuracy, and customer confidence.

3. During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Testing should be performed throughout the Software Development Life Cycle (SDLC) rather than only at the end.

- **Requirement phase:** Review requirements for errors.
- **Design phase:** Review system and database design.
- **Coding phase:** Perform unit testing.
- **Integration phase:** Test interaction between modules.
- **System testing:** Test the complete application.
- **Maintenance:** Perform regression testing.

Process discipline helps detect defects early and prevents integration and system-level failures

4. A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

These terms are related but different.

- **Error:** A human mistake made during development.
- **Defect:** A fault in the software caused by an error.
- **Failure:** The incorrect behavior observed when the software executes.

Example: A developer writes password == "12345" instead of checking the user's actual password. The developer's mistake is an **error**, the incorrect code is a **defect**, and allowing an unauthorized login is a **failure**.

Thus, **Error** → **Defect** → **Failure**.

5. A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

The principle of early testing states that testing should begin as early as possible in the SDLC. Testers can review requirements and design documents before coding begins. For example, if an ambiguous payment requirement is identified during requirement analysis, it can be corrected before implementation. Early detection reduces the cost, time, and effort required to fix defects later.

6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

A tester should participate in the **requirement analysis phase** and review requirements for ambiguity, missing information, inconsistency, and incorrect assumptions. For example, if a healthcare requirement says "patient data should be stored securely," the tester can ask for specific security requirements such as encryption and access control.

Early tester involvement prevents defects from entering later stages and improves overall software quality.

7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Defects can originate at any stage of the SDLC, especially due to unclear requirements, poor design, coding errors, or incorrect integration. For example, if a payroll requirement does not clearly define overtime calculation, developers may implement different interpretations. This produces incorrect salary calculations. Clear, complete, and testable requirements help prevent such defects.

8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

A crash at boundary input values is usually a boundary-related functional defect. For example, if an age field accepts values from 18 to 60, the tester should test:

- 17 – below boundary
- 18 – minimum
- 19 – just above minimum
- 59 – just below maximum
- 60 – maximum
- 61 – above maximum

Boundary Value Analysis would likely detect the problem before release.

9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

A defect repository stores information about previously identified defects, including their description, severity, cause, affected module, and resolution. Testers can study historical defects and create additional test cases for similar situations. For example, if previous releases had repeated payment failures, testers can include payment failure scenarios in regression testing. This prevents repeated defects and improves test coverage.

10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

An ATM withdrawal failure caused by incorrect transaction logic is a **functional defect** because the system does not perform the required function correctly.

Test cases should cover:

- Valid withdrawal
- Insufficient balance
- Maximum withdrawal limit
- Invalid amount
- Account balance update
- Cash dispensing

Proper functional and integration testing could have detected the defect before deployment.

11. A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Defect clustering means that a small number of modules often contain a large number of defects. If testing data shows that Modules A and B contain most defects, testers should give these modules higher priority. For example, if 70% of defects are found in the payment module, more testing resources should be allocated there. This improves testing efficiency and risk-based prioritization.

12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Slow performance under heavy user load is a performance/non-functional defect. The tester should perform load testing, stress testing, and scalability testing to identify the cause. For example, if a website becomes very slow when 10,000 users access it simultaneously, the problem may be related to database queries, server capacity, or poor resource management. Classification helps assign the problem to the appropriate team for resolution.

13. A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Historical defect data can be reused to create regression test cases. For example, if previous releases had defects in login, payment, and checkout, these areas should be included in every regression test cycle. This improves coverage of high-risk areas, prevents defect recurrence, and reduces testing effort because existing test cases can be reused.

14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Testers should communicate with developers early and participate in requirement and design discussions. During integration testing, testers verify whether modules exchange data correctly. For example, if the payment module sends an incorrect transaction status to the order module, early collaboration can help identify the expected interface and data format before integration. Early involvement reduces misunderstandings and integration defects.

15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Defect prevention focuses on avoiding defects rather than only finding and fixing them. The team can analyze recurring defects using Root Cause Analysis (RCA) and take preventive actions. For example, if incorrect calculations repeatedly occur because of unclear requirements, the team can introduce requirement reviews and better acceptance criteria. This reduces recurring defects and improves long-term software quality.

16. A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

The testing strategy should be standardized using:

- Test planning
- Requirement-based testing
- Systematic test case design
- Functional testing
- Compatibility testing
- Regression testing
- Defect tracking

For example, the mobile application should be tested on different devices, operating systems, screen sizes, networks, and user inputs. A structured approach improves **reliability, consistency, and stability**.

17. A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Testing is the process of finding defects or failures in software. Debugging is the process of locating the cause of a defect and correcting the code. Example: A tester finds that an ATM deducts ₹5,000 but dispenses ₹3,000. This is testing. The developer investigates the transaction calculation and fixes the code; this is debugging. Therefore, testing identifies problems, while debugging finds and fixes their causes.

18. An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

A cross-browser testing strategy verifies that the web application works correctly on different browsers such as Chrome, Firefox, Edge, and Safari.

Testers should verify:

- Page layout
- Buttons and links
- Forms
- Responsive design

19. cross-browser testing strategy. Explain how such issues can be effectively handled. A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Improper validation may allow incorrect or incomplete data to enter the system.

Testers should verify:

- Required fields
- Data formats
- Valid ranges
- Duplicate data
- Invalid inputs
- Boundary values

For example, a delivery distance should not accept a negative value. Proper validation testing prevents invalid data and improves system reliability.

20. A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Random testing provides unpredictable and incomplete coverage. Testing as an engineering activity requires **planned and systematic test design.**

Testers should analyze requirements and use techniques such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing

This ensures important scenarios are covered and increases the probability of finding defects.

21. A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Data inconsistency between healthcare modules may be caused by incorrect interfaces, database synchronization problems, incorrect data formats, or integration defects. Testers should perform integration testing by checking whether data sent from one module is correctly received and stored by another. For example, if a patient's updated phone number in the registration module is not reflected in the billing module, integration testing can identify the problem.

22. A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Testers should participate throughout the SDLC:

Requirements → Design → Development → Integration → System Testing → Maintenance

They review requirements, identify risks, design test cases, perform testing, report defects, and conduct regression testing. Early involvement helps identify problems before coding and reduces rework, development cost, and project delays.

23. A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

A system that fails during peak usage may have insufficient test coverage.

- Testers should include:
- Normal scenarios
- Boundary cases
- Negative cases
- High-load scenarios
- Concurrent users
- Failure and recovery scenarios

For example, an online ticket system should be tested with thousands of simultaneous users before a major sale. Better coverage helps identify critical failures before production.

24. A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

A defect repository should maintain complete information such as:

- Defect ID
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Status
- Assigned developer
- Resolution

Proper documentation makes defects easier to track, analyze, retest, and close. Historical records also help identify recurring problems.

25. A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

A banking system requires **systematic, risk-based, and continuous testing**. Testing should cover:

- Account creation
- Login/security
- Deposits
- Withdrawals
- Fund transfers
- Transaction limits
- Error handling
- Regression testing

For example, every fund transfer should verify debit, credit, transaction status, and balance update. Structured testing improves **accuracy, reliability, security, and customer trust**.

26. A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

Testing only normal inputs can miss important defects. Testers should include edge cases and boundary values. For example, if a system accepts transaction amounts from ₹100 to ₹50,000, test: ₹99, ₹100, ₹101, ₹49,999, ₹50,000, ₹50,001. Boundary testing helps detect incorrect comparisons, validation errors, crashes, and unexpected behavior.

27. A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Process-based testing uses a **standard testing process** across all modules. The organization should define common procedures for:

- Requirement review
- Test planning
- Test case design
- Test execution
- Defect reporting
- Retesting
- Regression testing
- Test closure

Standardization provides consistent testing quality across modules and reduces missed test scenarios.

28. A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

The web application should be tested with both valid and invalid inputs. Testers should check:

- Empty inputs
- Special characters
- Very long inputs
- Incorrect formats
- Negative values
- Duplicate values
- Unexpected data types

For example, a phone-number field should reject letters and invalid lengths. Proper input validation improves **robustness, security, and reliability**.

29. A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Defect leakage occurs when defects that should have been detected during testing reach the **production environment**.

Root causes may include:

- Poor test coverage
- Incomplete requirements
- Insufficient regression testing
- Lack of negative testing
- Inadequate environment testing

The team should perform root-cause analysis, improve test cases, automate regression tests where useful, and strengthen reviews. This reduces production defects.

30. A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

The testing strategy should focus on finding defects **as early as possible**. The team should:

- Review requirements early
- Perform design reviews
- Conduct unit testing
- Perform integration testing
- Use systematic test design
- Automate suitable regression tests
- Track defect trends

For example, detecting an incorrect requirement during analysis is much cheaper than discovering it after deployment

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

